

การสร้าง Account

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/accounts
- Body (Request):**

```
1 {
2   "accountNumber": "1234567890",
3   "ownerName": "วิภาวี ฤทธิหาญ",
4   "balance": 0
5 }
```
- Status:** 200 OK
- Response Time:** 342 ms
- Response Size:** 263 B
- Body (Response):**

```
1 {
2   "accountNumber": "1234567890",
3   "balance": 0.0,
4   "id": 1,
5   "ownerName": "วิภาวี ฤทธิหาญ"
6 }
```

ระบบสามารถสร้าง Account ได้สำเร็จ โดยส่งข้อมูล accountNumber, ownerName และ balance ผ่าน POST /accounts และระบบสร้าง id ให้กับ Account ใหม่ แสดงว่าข้อมูลสามารถถูกบันทึกลงฐานข้อมูล PostgreSQL ได้สำเร็จ

การฝากเงินสำเร็จ

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/accounts/1/deposit
- Body (Request):** A JSON object with the key "amount" and the value 1000.

```
1 {  
2   "amount": 1000  
3 }
```
- Status:** 200 OK
- Response Time:** 100 ms
- Response Size:** 188 B
- Body (Response):** A JSON object with the key "message" and the value "Deposit successful".

```
1 {  
2   "message": "Deposit successful"  
3 }
```

เมื่อเรียก POST /accounts/{id}/deposit และส่งจำนวนเงิน 1000 บาท ระบบตอบกลับด้วยข้อความ "Deposit successful" แสดงว่าการฝากเงินดำเนินการสำเร็จ โดยระบบมีการเพิ่มยอด balance และบันทึกประวัติการฝากเงินลงใน DepositTransaction

ตรวจสอบ Account

GET http://localhost:8080/accounts/1 Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 37 ms • 266 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "accountNumber": "1234567890",
3   "balance": 1000.0,
4   "id": 1,
5   "ownerName": "ริการี ฤทธิหาญ"
6 }
```

หลังจากฝากเงิน 1000 บาท เมื่อตรวจสอบ Account พบว่า balance เปลี่ยนจาก 0.0 เป็น 1000.0 แสดงว่ายอดเงินในบัญชีถูกเพิ่มขึ้นอย่างถูกต้อง

ตรวจสอบประวัติการฝากเงิน

lab9

1 SELECT * FROM deposit_transaction;

Data Output Messages Notifications

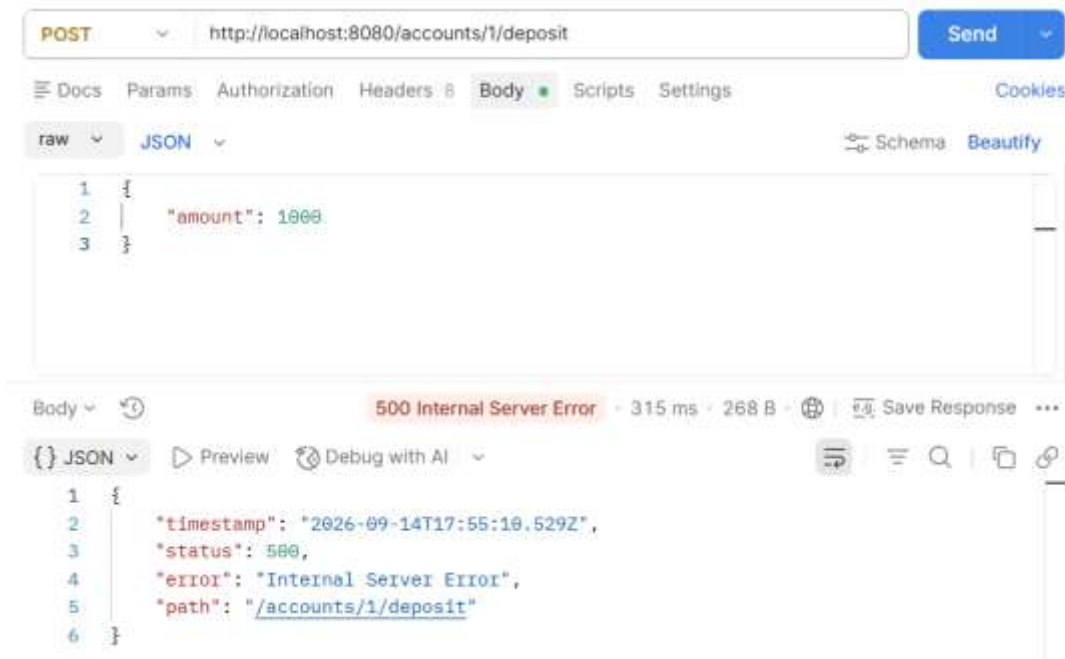
Showing rows: 1 to 1 Page No: 1 of 1

id [PK] bigint	amount double precision	account_id bigint
1	1000	1

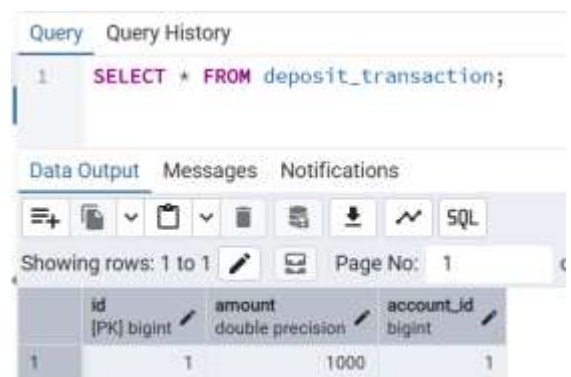
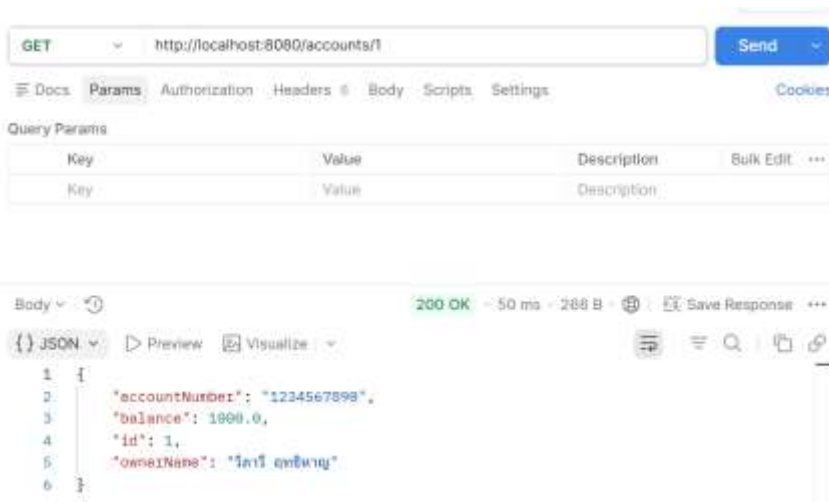
พบรายการฝากเงินจำนวน 1000 และมี account_id อ้างอิงไปยัง Account ที่ทำรายการ แสดงว่าบันทึกประวัติการฝากเงินสำเร็จ

ทดลอง Transaction Rollback

โดยการเพิ่ม `throw new RuntimeException("Test Rollback");` แล้ว



ระบบเกิด Error แต่เนื่องจากเมธอด `deposit()` มี `@Transactional` ระบบจึงทำการ ROLLBACK ข้อมูลทั้งหมดของ Transaction นั้นกลับคืน ส่งผลให้ `balance` ไม่เพิ่มขึ้น และไม่มี `DepositTransaction` รายการใหม่ถูกบันทึก



ทำการ comment @Transactional ไว้

POST `http://localhost:8080/accounts/1/deposit` Send

Docs Params Authorization Headers Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "amount": 1000
3 }
```

Body 500 Internal Server Error - 349 ms - 268 B Save Response

{ } JSON Preview Debug with AI

```
1 {
2   "timestamp": "2026-09-14T17:57:48.507Z",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/accounts/1/deposit"
6 }
```

GET `http://localhost:8080/accounts/1` Send

Docs Params Authorization Headers Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body 200 OK - 50 ms - 266 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "accountNumber": "1234567896",
3   "balance": 2000.0,
4   "id": 1,
5   "ownerName": "นาย อนุชา นามะ"
6 }
```

```
1 SELECT * FROM deposit_transaction;
```

Data Output Messages Notifications

1 to 2 Page No: 1 of 1

	amount	account_id	
1	1	1000	1
2	3	1000	1

เปรียบเทียบมี @Transactional กับไม่มี @Transactional

- เมื่อมี @Transactional การเพิ่ม balance และการบันทึก DepositTransaction จะถูกมองเป็นงานก้อนเดียวกัน หากเกิด Error ระบบจะ ROLLBACK การเปลี่ยนแปลงทั้งหมด
- เมื่อไม่มี @Transactional การ save() แต่ละครั้งไม่ได้อยู่ภายใต้ Transaction เดียวกัน เมื่อเกิด Error หลังจากการบันทึก ข้อมูลที่ถูกบันทึกไปแล้วจึงยังคงอยู่ในฐานข้อมูล ทำให้ balance และ DepositTransaction ถูกเปลี่ยนแปลง แม้ว่าการทำงานทั้งหมดจะจบด้วย Error

คำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

ตอบ : ใช้กำหนดให้คำสั่งหลาย ๆ คำสั่งในเมธอดทำงานเป็น Transaction เดียวกัน หากทุกขั้นตอนทำงานสำเร็จจะ COMMIT ข้อมูลลง Database แต่ถ้าเกิด Error ระบบจะ ROLLBACK

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

ตอบ : เพราะการฝากเงินต้องทำงานอย่างน้อย 2 อย่าง คือ เพิ่ม balance ของ Account และบันทึก DepositTransaction ทั้งสองขั้นตอนควรสำเร็จพร้อมกัน หากขั้นตอนใดเกิด Error ต้องยกเลิกการเปลี่ยนแปลงทั้งหมด เพื่อป้องกันข้อมูลไม่สอดคล้องกัน

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

ตอบ : COMMIT เป็นการยืนยันการเปลี่ยนแปลงและบันทึกข้อมูลลง Database อย่างถาวร ROLLBACK คือการยกเลิกการเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction และทำให้ข้อมูลกลับไปเป็นสถานะก่อนเริ่ม Transaction

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

ตอบ : มีความสัมพันธ์แบบ One-to-Many โดย Account 1 บัญชีสามารถมี DepositTransaction ได้หลายรายการ และ DepositTransaction แต่ละรายการจะอ้างอิง Account หนึ่งบัญชีผ่าน Foreign Key ชื่อ account_id

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ตอบ : ข้อมูลที่ถูกเปลี่ยนแปลงภายใน Transaction จะถูก ROLLBACK ทั้งหมด ทำให้การเพิ่ม balance และการบันทึก DepositTransaction ที่เกิดขึ้นก่อน Error ไม่ถูกบันทึกลง Database

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ตอบ : เมื่อไม่มี @Transactional และเกิด Error หลังจากคำสั่ง save() ข้อมูลที่ถูกบันทึกไปแล้วอาจยังคงอยู่ใน Database เนื่องจากไม่ได้ถูกควบคุมเป็น Transaction เดียวกัน ดังนั้น balance และ DepositTransaction ที่บันทึกก่อนเกิด Error จึงยังสามารถคงอยู่ได้ แตกต่างจากกรณีที่มี @Transactional ซึ่งจะ ROLLBACK ข้อมูลทั้งหมด

